Commit first, then let Claude move things

Your own project as a git repository, and on GitHub

Florian Gärtner

Where we are today

	What Day 1 taught
	What Claude needs in every session
	What Claude needs only sometimes
	Your own skill
	Lunch
	A history you can go back to
	Where does this file live?
→	**Commit first, then let Claude move things. Then GitHub** ← you are here
	Wrap, done at 17:00

Learning objectives

After this block you can:

1. **Run** `/newproject` on one of your projects, and read what Claude proposes before it changes anything.
2. **Set up** one of your projects as a git repository, in a folder no cloud service syncs, with a commit before Claude moves anything.
3. **Check** the result in what Claude shows you: the state, the files git tracks, and one run of your scripts if you have any.
4. **Push** your project to a private repository on GitHub, with the account and the key you set up yourself.

Your turn

Step 1: run `/newproject`

Back in your project's session (`/cd` and its path), type `/newproject`.

Claude says what it sees, a new project or one with files already, and asks before it changes anything. Then it asks you a few questions, one at a time, and proposes what to set up or fix.

Without the kit, ask Claude for a plan from the course's slides instead, with `/plan` in front:

> Download the slides of blocks 2_5 and 2_6 from `github.com/F-Gaertner/course_ggs_2026_material` into a new folder `course_slides` in my home folder. Read them. How would you set up this project as a git repository, following them?

What `/newproject` sets up

```
my_project/                  in ~/repos, not in a cloud folder
  CLAUDE.md                  what Claude needs to know about this project
  claude/                    the handover, open issues, the log, decisions
  analysis/
    0_shared/                setup: packages, paths, shared helpers
    1_raw_data/              a copy of your data, which you put in
    2_interim_data_steps/    data between steps
    3_clean_data/            the data the analysis uses
    4_scripts/               the pipeline: load, clean, analyse, tables, plots
    5_tables/  6_plots/      what the scripts produce
    7_r_objects/             saved results (Python: 7_objects/)
    Data_Analysis_Main_File_OPEN_ME_FIRST.R / .py   runs every script in order
  .gitignore                 what git leaves out: data and everything generated
  .git/                      the git repository, with a first commit
```

In R or in Python, as you choose. The scripts run from the start, and stop with a message until your data is in `1_raw_data/`.

Make it yours

The layout is a starting point, not a rule. Change what does not fit your work:

- rename, add or drop a stage, or split a script that grows too long
- switch from R to Python, or mix them
- drop a folder you will never use

Ask Claude to make the change, and to update the main file and `CLAUDE.md` with it, so the next session knows the new shape. **The pipeline is yours; the kit only gave it a start.**

Step 2: read what Claude proposes, then approve it

Read what Claude proposes, and ask about every step you do not understand. Then approve it. With `/newproject` the project ends up the kit's way: folders, `.gitignore`, a lock on the raw data, git. If Claude offers a GitHub repository now, answer *not yet*: Steps 3 to 7 set it up.

Is there a commit before the first file moves? If not, ask for one. That commit is your way back if the moves go wrong.

If the repository goes into a new folder, move the session there at the end: `/cd` and the path Claude names.

Commit first, then let Claude move things.

Your raw data: the original and a copy

Never work on the raw data, always on a copy of it.

- **The original** lives in a cloud folder, such as JLU's Nextcloud, shared with your coauthors. That is its backup: git leaves your raw data out, so a lost laptop would lose the only copy.
- **The copy** goes into your project's `1_raw_data/`, and the scripts read only that. You put it there yourself: the kit's settings refuse Claude's own edits in that folder. With `/newproject`, git leaves the folder out, so personal or licensed data can sit there without reaching git or GitHub.

A request or a script that goes wrong then changes the copy, and the original is still there. When the original changes, copy it in again.

Check-in

`/newproject` is a long procedure, and it has stopped halfway before. When Claude says it is done, ask:

> Check this project against your new_project.md. Is anything missing?

Then:

> Show me the state. List the files git tracks, and the files it leaves out.

Everyone has a commit from before the moves, no data among the tracked files, and none of the files you write among the files left out?

If the project broke

What you see	What to do
your scripts cannot find the data	ask Claude to run them and fix the path the error names
a LaTeX paper cannot find its table	ask Claude to point its `\input` line at the new place
the old scripts sit beside the new ones	ask Claude to remove the old ones with `git rm`

If git looks wrong

What you see	What to do
a data file among the tracked files	ask Claude to untrack it (`git rm --cached`), fix the `.gitignore`, and commit
a file that should be ignored shows in the state	ask Claude which line of `.gitignore` lets it in
the reorganisation went wrong	go back to the first commit (next slide)

Back to the first commit

Ask:

> Put the whole project back as it was at the first commit, as a new commit. Keep the later commits in the history.

Your files are as they were at the first commit, and the reorganisation stays in the history if you want it back. Files git ignores, like your outputs, stay.

Now a copy on GitHub

Your project has a history now, but only on this laptop. Steps 3 to 7 put a copy on GitHub: the remote from block 2_5, private, so only you and the people you invite can see it.

Three of the steps only you can do, in the browser, because Claude has no login to your account: make the account, give it your laptop’s key, and make an empty repository. Claude makes the key, tests it, connects your project and pushes.

The account and the key you set up once per laptop. After that, each new project needs an empty repository and one request.

Step 3: an account

No account yet? Sign up at github.com. It is free, and GitHub sends a code to your email.

Then log in, in the browser you keep open beside Claude, because Steps 5 and 6 happen on the website.

Tell me your username, and I invite you to the course's organization on GitHub, which holds the kit and, after today, the course material. Accept the invitation in the email GitHub sends you.

Step 4: a key

An **SSH key** is how GitHub recognises your laptop when git talks to it, in place of a password. It is a pair of files on your laptop:

- the **private half** stays there. Whoever has this file can push as you.
- the **public half** is what you give GitHub, which checks each connection against it.

Ask:

> Make an SSH key for GitHub, without a passphrase, and open its public half in an editor.

Step 5: the key on GitHub

Copy the public half, one line that starts with `ssh-ed25519`. On GitHub: your picture → **Settings** → **SSH and GPG keys** → **New SSH key**. Paste it under **Key**, then **Add SSH key**.

Then ask:

> Test the connection to GitHub.

If Claude reports your username, the key works.

Lost the laptop? Delete the key on GitHub, on the same **SSH and GPG keys** page.

Step 6: an empty repository

On GitHub: **+** (top right) → **New repository**.

- a name, for example your project folder's
- visibility **Private** (GitHub preselects Public)
- leave README, `.gitignore` and licence off

Create repository, then copy the address that starts with `git@github.com:`.

Step 7: the push

Ask, with that address:

> Add `<url>` as the remote, and push main to it.

With your kit, Claude Code asks you before the push, in Auto too. Approve it.

Reload the repository's page, and check that it shows the files you write, and no data.

Material you hold for others: JLU GitLab

JLU's own GitLab, `gitlab.ub.uni-giessen.de`, takes the same steps with your JLU ID. Your key goes into your GitLab profile, then you make a new project and send Claude the same request.

SSH works there only through the university VPN. Off campus, the push needs a project access token instead.

JLU's terms allow it for data with normal protection needs.

Done when

After the push, check what `/newproject` cannot check for you:

- your project's page on GitHub is private, and shows your files but no data
- the scripts run from the raw data to the tables

If Claude moved scripts you already had

- the old copies are gone
- in LaTeX, the `\input` lines point at the tables' new place

Then share: one thing that broke, and how you found it.

What to take away

1. For every new project, give Claude what it needs to know, and ask for its plan before it changes anything. With your kit, `/newproject` does both.
2. Commit first, then let Claude move things.
3. Never work on the raw data, always on a copy of it.
4. Check in what Claude shows you: the state, the files git tracks, and one run of your scripts if you have any.
5. GitHub needs three things only you can do: an account, your key pasted in, and an empty repository. After that, a push is one request.